FUNDAMENTALS OF DATA SCIENCE

Combined Repository – Python Lab Experiments

Compiled from the uploaded GitHub repository ZIP. Notebook cells, code, available outputs, and the accompanying CSV data file are included.

Source files: 21 Jupyter notebooks and 1 CSV file

Experiment / Notebook 1: 01a.ipynb

Code cell 1

```
""" [Q] Analyze the trend of data science job postings over the last decade for the
given data on the number of data science job postings each year. Use
pandas for data manipulation and matplotlib /seaborn for visualization.

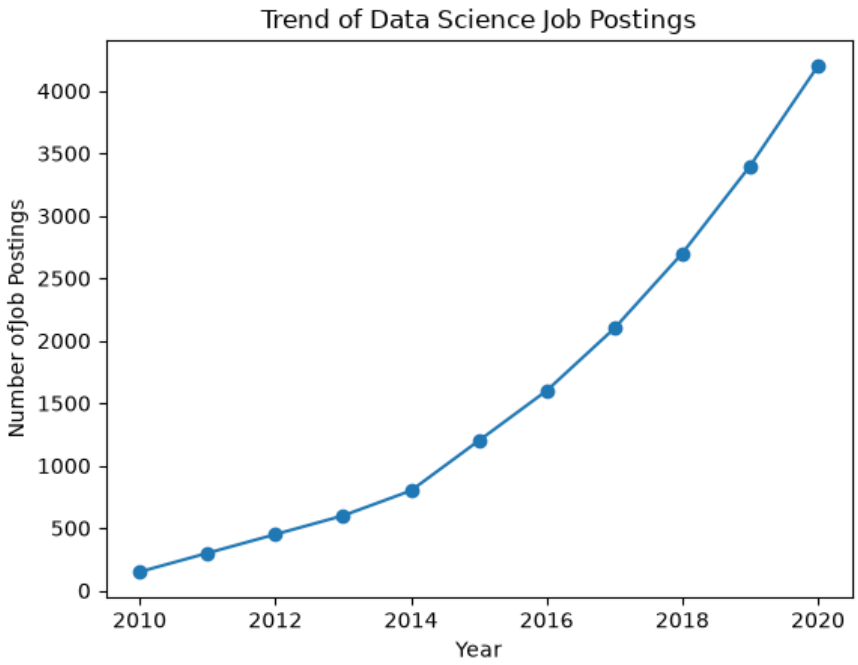
    Description: Use web scraping (e.g., BeautifulSoup) or APIs (e.g., LinkedIn API) to gather
data on the number of data science job postings each year. Use pandas for data manipulation
and matplotlib/seaborn for visualization.
    """
import pandas as pd
import matplotlib.pyplot as plt
data = {'Year': list(range(2010, 2021)),
'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]}

df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number ofJob Postings')
plt.show()

"""
Sample Data Input:

Year =2010, 2021
Job Postings=150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200
"""
```

Output image:



Output 1

```
<Figure size 640x480 with 1 Axes>
```

Output 2

```
'\nSample Data Input:\n\nYear =2010, 2021\nJob Postings=150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200\n'
```

Experiment / Notebook 2: 01b.ipynb

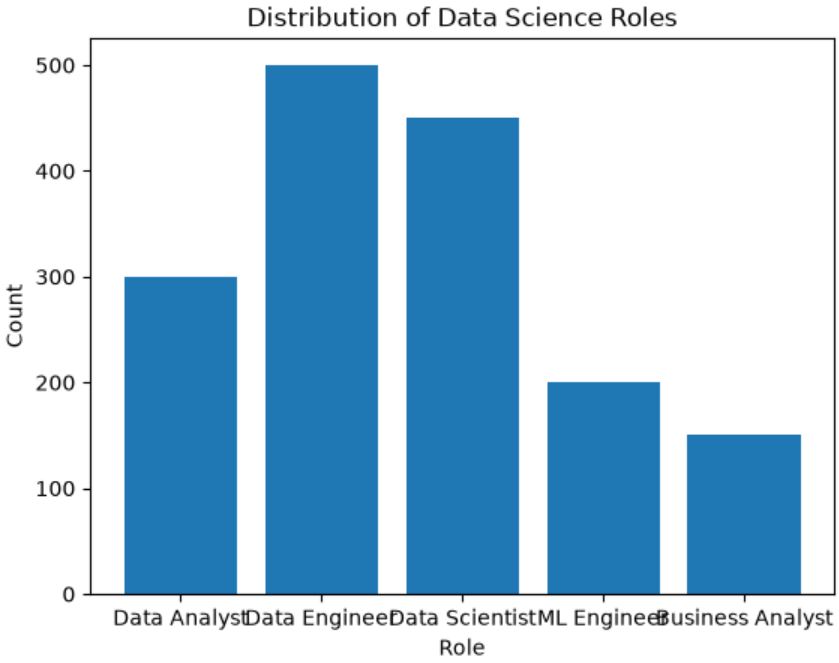
Code cell 1

```
"""
Exp No:1.b Analyze and visualize the distribution of various data science roles (Data
Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

Description: Use a dataset of job postings and categorize them into different roles. Visualize
the distribution using pie charts or bar plots.

Sample Data Input:
roles = Data Analyst, Data Engineer, Data Scientist, ML Engineer, Business Analyst.
counts = 300, 500, 450, 200, 150.
   """
import matplotlib.pyplot as plt
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer','Business Analyst']
counts = [300, 500, 450, 200, 150]
plt.bar(roles, counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.show()
```

Output image:



Output 1

```
<Figure size 640x480 with 1 Axes>
```

Code cell 2

```
# (empty code cell)
```

Experiment / Notebook 3: 01c.ipynb

Code cell 1

```
"""
Exp No:1.c Conduct an experiment to differentiate Structured , Un-structured and Semi
structured data based on data sets given.
Description: Create small datasets for each type and explain their characteristics.
"""
import pandas as pd
# Structured data example
structured_data = pd.DataFrame({
'ID': [1, 2, 3],
'Name': ['Alice', 'Bob', 'Charlie'],
'Age': [25, 30, 35]
})
print("Structured Data:\n", structured_data)

# Unstructured data example
unstructured_data = "This is an example of unstructured data. It can be a piece of text, an image, or a video file."
print("\nUnstructured Data:\n", unstructured_data)

# Semi-structured data example (JSON)
semi_structured_data = {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
print("\nSemi-structured Data:\n", semi_structured_data)
```

Output 1

```
Structured Data:
    ID     Name  Age
0   1    Alice   25
1   2      Bob   30
2   3  Charlie   35

Unstructured Data:
 This is an example of unstructured data. It can be a piece of text, an image, or a video file.

Semi-structured Data:
 {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
```

Code cell 2

```
# (empty code cell)
```

Experiment / Notebook 4: 01d.ipynb

Code cell 1

```
"""
Exp No:1.d Conduct an experiment to encrypt and decrypt given sensitive data.
Description: Use the cryptography library to encrypt and decrypt a piece of data.
"""
# Generate key and encrypt data
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(b"Rajalakshmi Engineering College")
token
b'...'
f.decrypt(token)
b'Rajalakshmi Engineering College'
key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Rajalakshmi Engineering College."
cipher_text = cipher_suite.encrypt(plain_text)
# Decrypt data
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

Output 1

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABqx0oBIOS7pKmPkxiIcPnGptWSQGhm15CeU9H-zZvRTgAq4zT_I7MTOb4sWaemmj2_qhlAW4s3ygVWmVWHqU03FEZMlwzY6ExMpx
Decrypted Data: b'Rajalakshmi Engineering College.'
```

Code cell 2

```
# (empty code cell)
```

Experiment / Notebook 5: 02.ipynb

Code cell 1

```
"""
Exp No:2 Upload and Analyze the data set given in csv format and perform data
preprocessing and visualization.
Description: Use sample data set sales-data.csv.
"""
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
# Load the data into a pandas DataFrame
file_path = r"C:\Users\Pradeep\Documents\WORKS\FDS\sales_data.csv"
df = pd.read_csv(file_path)
# Display the first few rows of the DataFrame
print(df.head())
# Check for missing values
print(df.isnull().sum())
# Fill or drop missing values if necessary
df['Sales'] = df['Sales'].fillna(df['Sales'].mean())
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
# Summary statistics
print(df.describe())
# Group by product and calculate the total sales and quantity
product_summary = df.groupby('Product').agg({
'Sales': 'sum',
'Quantity': 'sum'
}).reset_index()
print(product_summary)

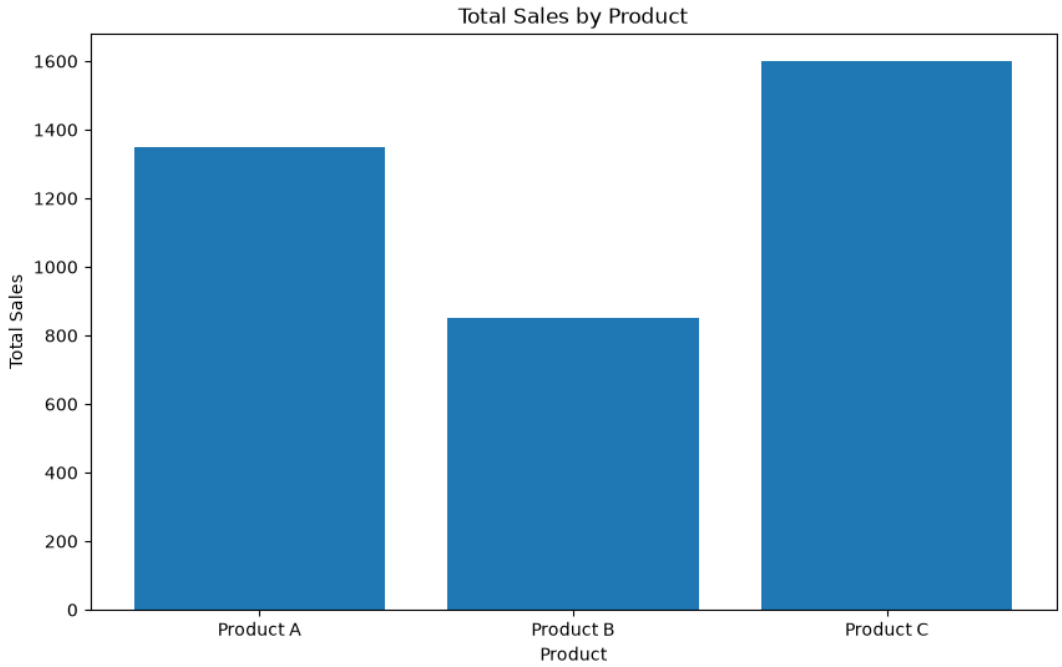
# Bar plot of total sales by product
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
# Line plot of sales over time
df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y')
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('SalesOver Time')
plt.show()
# Pivot table to analyze sales by region and product
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product',
aggfunc=np.sum, fill_value=0)
print(pivot_table)
# Correlation matrix
correlation_matrix = df.corr(numeric_only=True)
print(correlation_matrix)
# Heatmap of the correlation matrix
import seaborn as sns
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

Output 1

```
         Date    Product  Sales  Quantity Region
0  01-01-2023  Product A    200         4  North
1  02-01-2023  Product B    150         3  South
2  03-01-2023  Product A    220         5  North
3  04-01-2023  Product C    300         6   East
4  05-01-2023  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
std     64.031242   1.746425
```

```
min    150.000000   3.000000
25%    187.500000   4.000000
50%    225.000000   5.500000
75%    302.500000   7.000000
max    340.000000   8.000000
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```

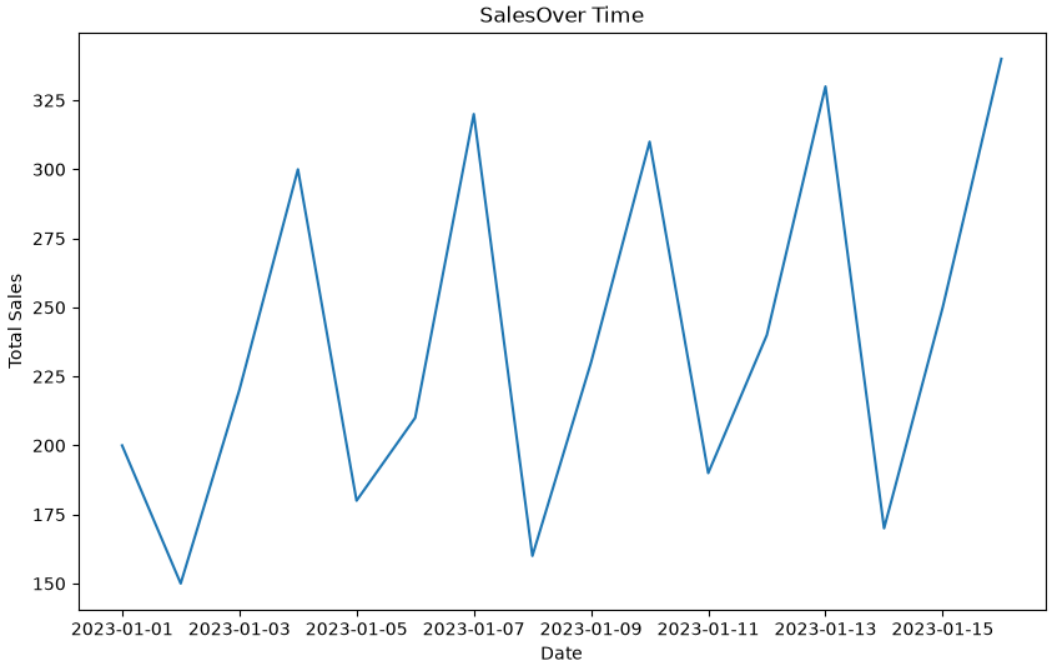
Output image:



Output 2

```
<Figure size 1000x600 with 1 Axes>
```

Output image:



Output 3

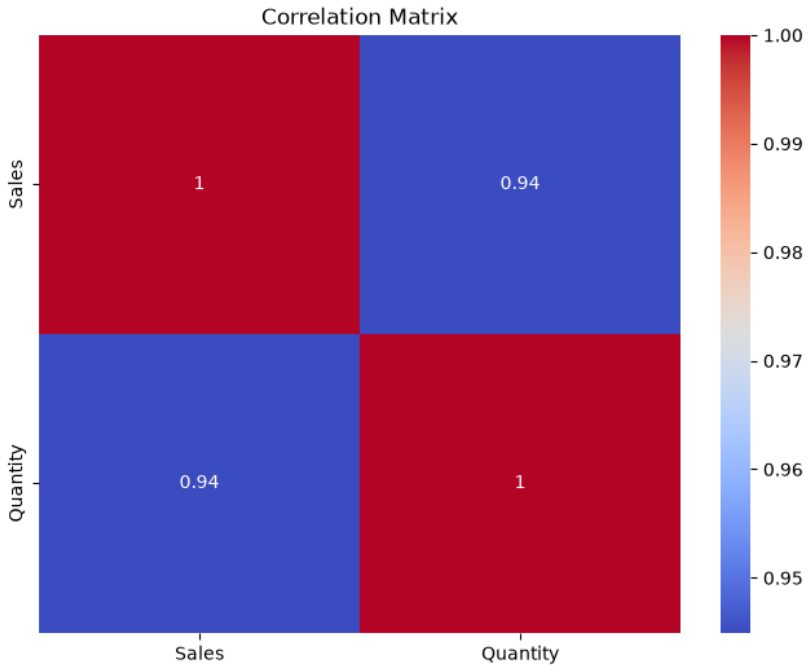
```
<Figure size 1000x600 with 1 Axes>
```

Output 4

```
Product  Product A  Product B  Product C
Region
East             0          0       1600
```

```
North         1350          0          0
South            0        480          0
West             0        370          0
              Sales  Quantity
Sales      1.000000  0.944922
Quantity   0.944922  1.000000
```

Output image:



Output 5

```
<Figure size 800x600 with 2 Axes>
```

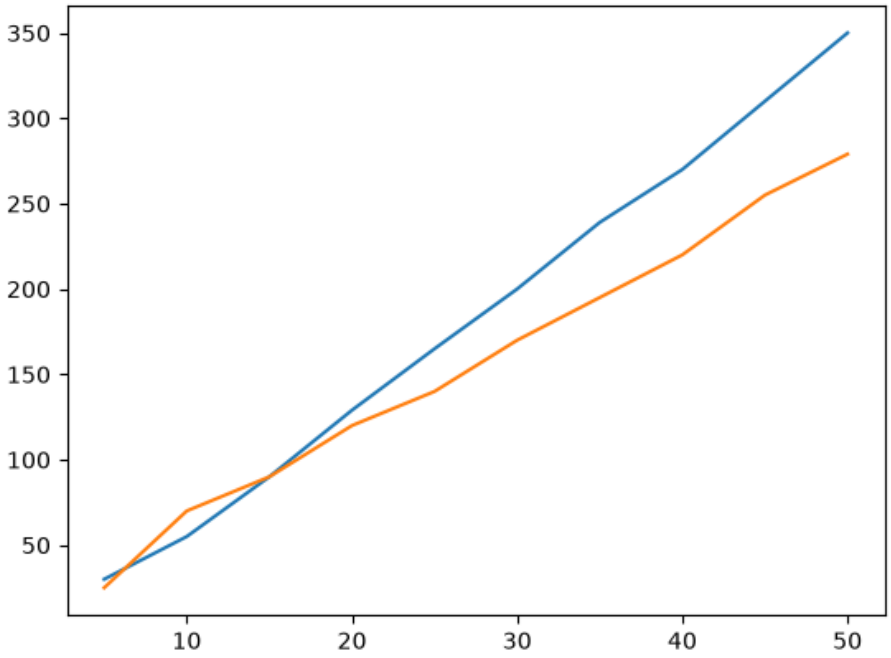
Experiment / Notebook 6: 03a.ipynb

Code cell 1

```
"""
Exp No:3.a Conduct an experiment to show data visualization using line plot
.
Description: Take any sample data either through csv file or data fetched directly through
code.
"""

import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```

Output image:



Output 1

```
<Figure size 640x480 with 1 Axes>
```

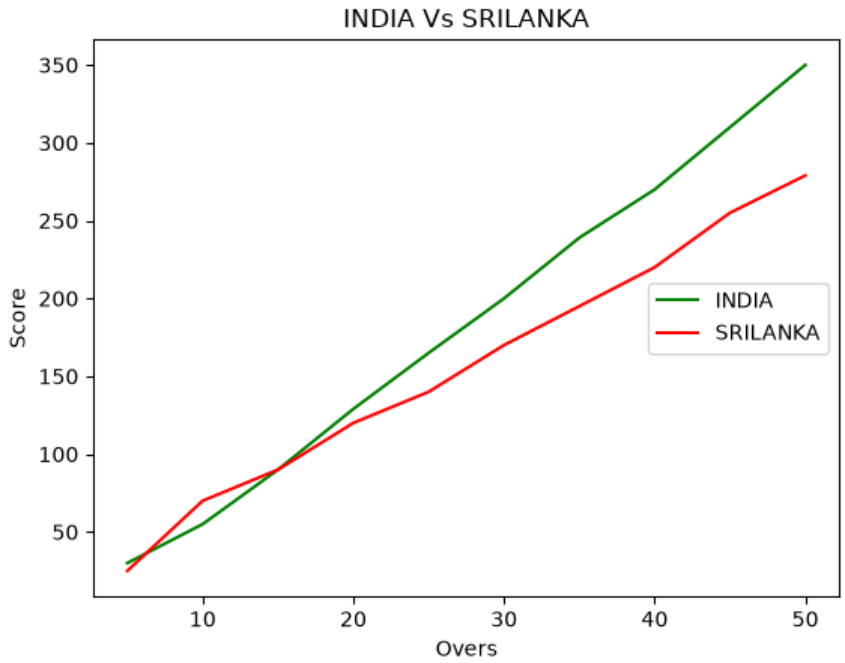
Output 2

```
C:\Users\Pradeep\AppData\Local\Temp\ipykernel_30132\1548337705.py:18: UserWarning: No artists with labels found to put in le
  cricket.legend()
```

Output 3

```
<matplotlib.legend.Legend at 0x1c8e18ed810>
```

Output image:



Output 4

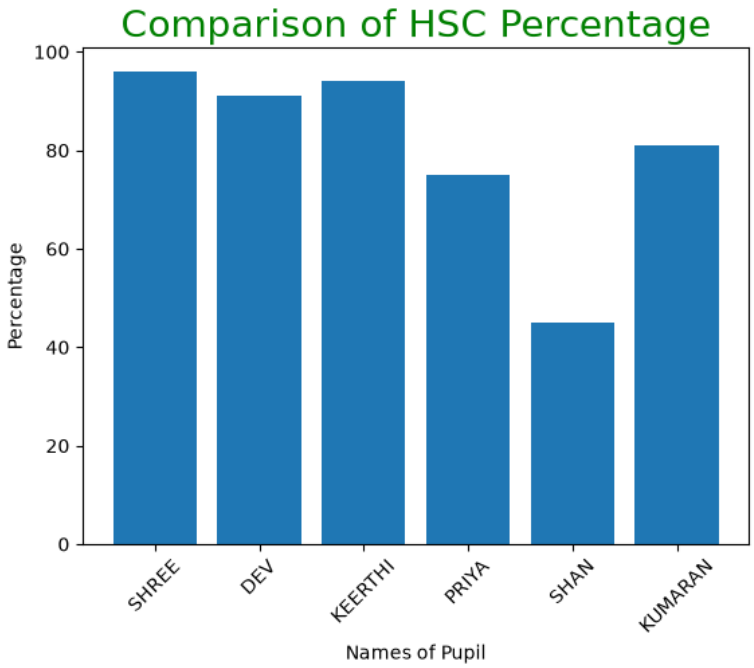
```
<Figure size 640x480 with 1 Axes>
```

Experiment / Notebook 7: 03b.ipynb

Code cell 1

```
"""
Exp No:3.b Conduct an experiment to show data visualization using bar chart
.
Description: Take any sample data either through csv file or data fetched directly through
code.
"""
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```

Output image:



Output 1

```
<Figure size 640x480 with 1 Axes>
```

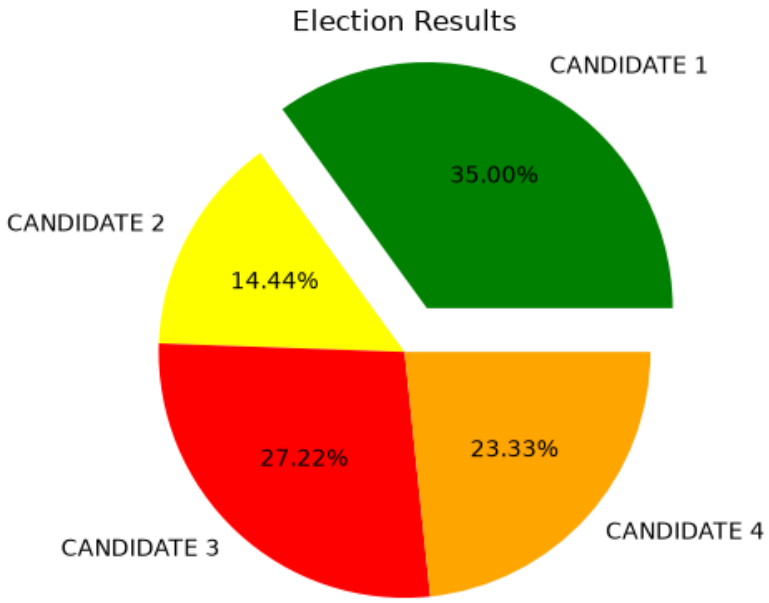
Experiment / Notebook 8: 03c.ipynb

Code cell 1

```
"""
Exp No:3.c Conduct an experiment to show data visualization using pie chart
.Description: Take any sample data either through csv file or data fetched directly through code.
"""

import matplotlib.pyplot as election
# Election data
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)
# Plotting the pie chart
election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%')
election.title('Election Results')
election.show()
```

Output image:



Output 1

```
<Figure size 640x480 with 1 Axes>
```

Experiment / Notebook 9: 04.ipynb

Code cell 1

```
"""
Exp No:4 To Count the frequency of occurrence of a word in a body of text is often

needed during text processing.
Description: Import the word_tokenize function and gutenberg.
"""
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
# Download the gutenberg corpus if not already installed
nltk.download('gutenberg')
nltk.download('punkt') # Ensure the punkt tokenizer models are also downloaded
# Load the text from the gutenberg corpus
sample = gutenberg.raw("austen-emma.txt") # Change to an existing text from the corpus
# Tokenize the sample text
token = word_tokenize(sample)
# Create a list of the first 50 tokens
wlist = []
for i in range(50):
    wlist.append(token[i])
# Calculate the frequency of each word in the list
wordfreq = [wlist.count(w) for w in wlist]
# Print the word-frequency pairs
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

Output 1

```
[nltk_data] Downloading package gutenberg to
[nltk_data]     C:\Users\Pradeep\AppData\Roaming\nltk_data...
[nltk_data]   Package gutenberg is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Pradeep\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

Output 2

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1
```

Code cell 2

```
import sys
!{sys.executable} -m pip install nltk
```

Output 1

```
Collecting nltk
  Downloading nltk-3.10.3-py3-none-any.whl.metadata (3.2 kB)
Requirement already satisfied: defusedxml in C:\Users\Pradeep\AppData\Local\Programs\Python\Python313\Lib\site-packages (fro
Requirement already satisfied: click in C:\Users\Pradeep\AppData\Local\Programs\Python\Python313\Lib\site-packages (from nlt
Requirement already satisfied: joblib in C:\Users\Pradeep\AppData\Local\Programs\Python\Python313\Lib\site-packages (from nl
Collecting regex>=2021.8.3 (from nltk)
  Downloading regex-2026.9.29-cp313-cp313-win_amd64.whl.metadata (41 kB)
Requirement already satisfied: tqdm in C:\Users\Pradeep\AppData\Local\Programs\Python\Python313\Lib\site-packages (from nltk
Requirement already satisfied: colorama in C:\Users\Pradeep\AppData\Local\Programs\Python\Python313\Lib\site-packages (from
Downloading nltk-3.10.3-py3-none-any.whl (1.8 MB)
   ---------------------------------------- 0.0/1.8 MB ? eta -:--:--
   ---------------------- ---------------- 1.0/1.8 MB 6.1 MB/s eta 0:00:01
   ---------------------------------------- 1.8/1.8 MB 5.8 MB/s  0:00:00
Downloading regex-2026.9.29-cp313-cp313-win_amd64.whl (280 kB)
Installing collected packages: regex, nltk

   ------------------- -------------------- 1/2 [nltk]
   ------------------- -------------------- 1/2 [nltk]
   ------------------- -------------------- 1/2 [nltk]
   ------------------- -------------------- 1/2 [nltk]
   ------------------- -------------------- 1/2 [nltk]
   ------------------- -------------------- 1/2 [nltk]
   ------------------- -------------------- 1/2 [nltk]
   ------------------- -------------------- 1/2 [nltk]
   ------------------- -------------------- 1/2 [nltk]
   ------------------- -------------------- 1/2 [nltk]
   ------------------- -------------------- 1/2 [nltk]
   ------------------- -------------------- 1/2 [nltk]
   ---------------------------------------- 2/2 [nltk]

Successfully installed nltk-3.10.3 regex-2026.9.29
```

Code cell 3

```
import nltk
nltk.download('punkt_tab')
```

Output 1

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Pradeep\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
```

Output 2

```
True
```

Experiment / Notebook 10: 05.ipynb

Experiment 5: Data Collection and Initial Exploration

Executed notebook with output.

Code cell 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
rng=np.random.default_rng(42)
df=pd.DataFrame({'Pregnancies':rng.integers(0,10,30),'Glucose':rng.integers(70,190,30),'BMI':np.round(rng.uniform(18,45,30),
df['Outcome']=((df.Glucose>140)&(df.BMI>28)).astype(int)
print('First five rows:'); display(df.head())
print('Dataset information:'); df.info()
print('Summary statistics:'); display(df.describe())
print('Missing values:'); print(df.isna().sum())
df.hist(figsize=(9,6),edgecolor='black'); plt.suptitle('Diabetes Dataset - Initial Exploration'); plt.tight_layout(); plt.sh
```

Output 1

```
First five rows:
```

Output 2

	Pregnancies	Glucose	BMI	Age	Outcome
0	0	124	38.1	42	0
1	7	97	44.1	53	0
2	6	81	26.8	53	0
3	4	136	28.0	44	0
4	4	176	30.7	62	1

Output 3

```
Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Pregnancies  30 non-null     int64
 1   Glucose      30 non-null     int64
 2   BMI          30 non-null     float64
 3   Age          30 non-null     int64
 4   Outcome      30 non-null     int64
dtypes: float64(1), int64(4)
memory usage: 1.3 KB
Summary statistics:
```

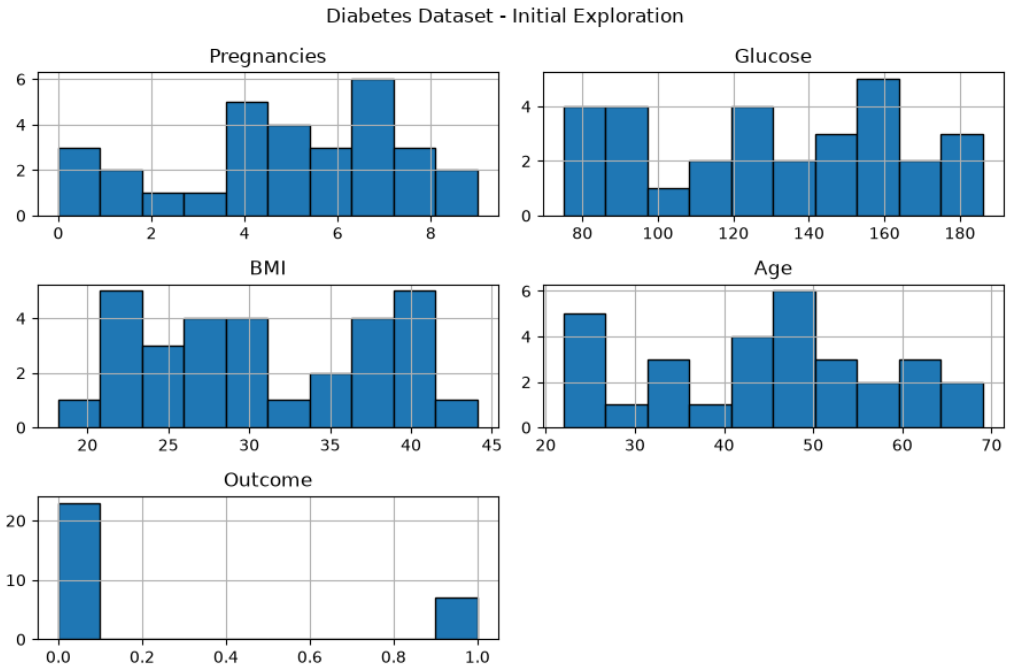
Output 4

	Pregnancies	Glucose	BMI	Age	Outcome
count	30.000000	30.000000	30.000000	30.000000	30.000000
mean	4.966667	130.100000	30.973333	44.800000	0.233333
std	2.697168	34.524704	7.287113	13.966462	0.430183
min	0.000000	75.000000	18.200000	22.000000	0.000000
25%	4.000000	98.500000	24.525000	34.250000	0.000000
50%	5.000000	132.000000	30.550000	48.000000	0.000000
75%	7.000000	159.750000	36.975000	53.000000	0.000000
max	9.000000	186.000000	44.100000	69.000000	1.000000

Output 5

```
Missing values:
Pregnancies    0
Glucose        0
BMI            0
Age            0
Outcome        0
dtype: int64
```

Output image:



Output 6

```
<Figure size 900x600 with 6 Axes>
```

Experiment / Notebook 11: 06.ipynb

Experiment 6: Time Series Analysis of Website Traffic

Executed notebook with output.

Code cell 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
from statsmodels.tsa.seasonal import seasonal_decompose
rng=np.random.default_rng(7); dates=pd.date_range('2025-01-01',periods=120,freq='D'); t=np.arange(120)
views=1200+4*t+180*np.sin(2*np.pi*t/7)+rng.normal(0,45,120); views[[35,92]]+=[700,-500]
df=pd.DataFrame({'Page_Views':views,'Unique_Visitors':views*.62+rng.normal(0,25,120),'Bounce_Rate':42+4*np.sin(2*np.pi*t/7)
df['Moving_Average']=df.Page_Views.rolling(7).mean(); df['Z_score']=(df.Page_Views-df.Page_Views.mean())/df.Page_Views.std()
print('Detected anomalies:'); display(anomalies[['Page_Views','Z_score']])
plt.figure(figsize=(11,4)); plt.plot(df.index,df.Page_Views,label='Page Views'); plt.plot(df.index,df.Moving_Average,label='
seasonal_decompose(df.Page_Views,model='additive',period=7).plot(); plt.tight_layout(); plt.show()
```

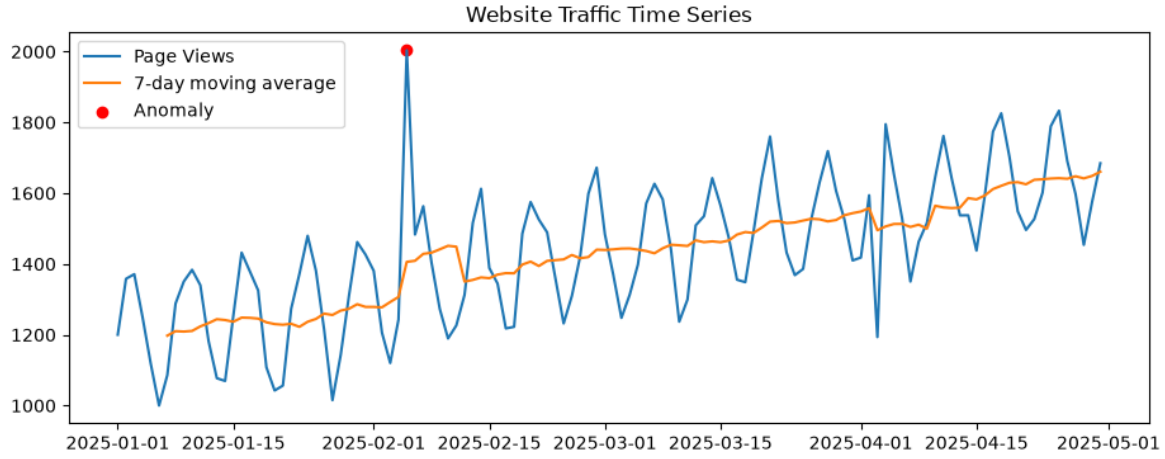
Output 1

```
Detected anomalies:
```

Output 2

```
            Page_Views  Z_score
2025-02-05  2003.66094  2.87361
```

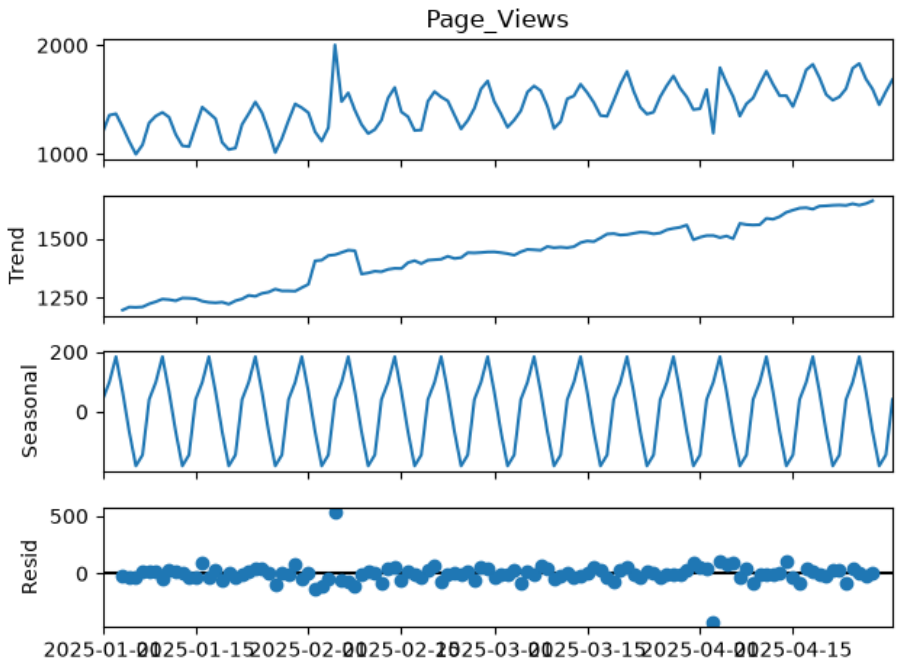
Output image:



Output 3

```
<Figure size 1100x400 with 1 Axes>
```

Output image:



Output 4

```
<Figure size 640x480 with 4 Axes>
```

Experiment / Notebook 12: 07.ipynb

Experiment 7: Random Sampling and Sampling Distribution

Executed notebook with output.

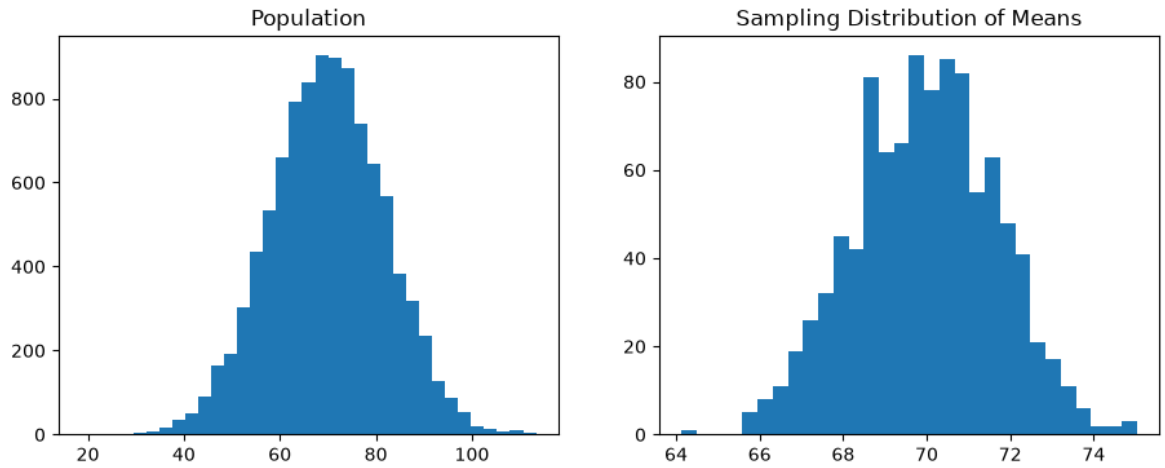
Code cell 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
rng=np.random.default_rng(21); population=rng.normal(70,12,10000); sample=rng.choice(population,50,replace=False); means=[rn
print(f'Population mean: {population.mean():.2f}'); print(f'Sample mean: {sample.mean():.2f}'); print(f'Mean of sampling dis
fig,ax=plt.subplots(1,2,figsize=(11,4)); ax[0].hist(population,bins=35); ax[0].set_title('Population'); ax[1].hist(means,bin
```

Output 1

```
Population mean: 69.96
Sample mean: 68.07
Mean of sampling distribution: 69.97
```

Output image:



Output 2

```
<Figure size 1100x400 with 2 Axes>
```

Experiment / Notebook 13: 08.ipynb

Experiment 8: Z Test

Executed notebook with output.

Code cell 2

```
import numpy as np
from scipy.stats import norm
sample=np.array([52,49,54,51,53,50,55,52,48,54]); mu=50; sigma=4
z=(sample.mean()-mu)/(sigma/np.sqrt(len(sample))); p=2*(1-norm.cdf(abs(z)))
print(f'Sample mean: {sample.mean():.2f}'); print(f'Z statistic: {z:.4f}'); print(f'P value: {p:.4f}'); print('Decision at a
```

Output 1

```
Sample mean: 51.80
Z statistic: 1.4230
P value: 0.1547
Decision at alpha=0.05: Fail to reject H0
```

Experiment / Notebook 14: 09.ipynb

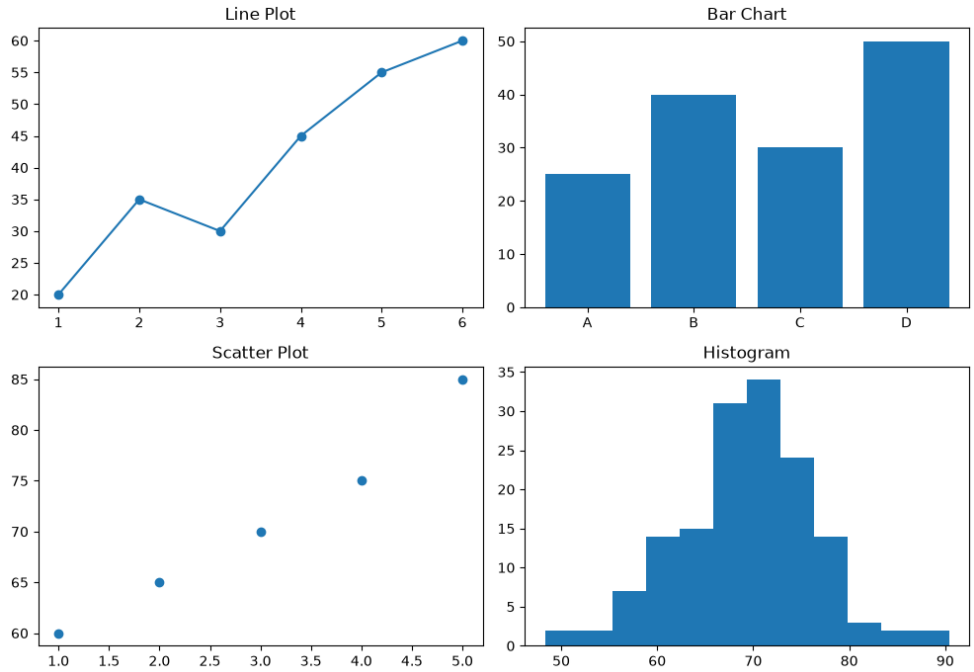
Experiment 9: Matplotlib Library Use Cases

Executed notebook with output.

Code cell 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
x=np.arange(1,7); fig,ax=plt.subplots(2,2,figsize=(10,7)); ax[0,0].plot(x,[20,35,30,45,55,60],marker='o'); ax[0,0].set_title
```

Output image:



Output 1

```
<Figure size 1000x700 with 4 Axes>
```

Experiment / Notebook 15: 10.ipynb

Experiment 10: Array Functions

Executed notebook with output.

Code cell 2

```
import numpy as np
a=np.array([1,2,3,4,5,6]); b=a.reshape(2,3)
print('Original:',a); print('Reshaped:\n',b); print('Zeros:\n',np.zeros((2,3),dtype=int)); print('Range:',np.arange(0,10,2)
```

Output 1

```
Original: [1 2 3 4 5 6]
Reshaped:
 [[1 2 3]
 [4 5 6]]
Zeros:
 [[0 0 0]
 [0 0 0]]
Range: [0 2 4 6 8]
Squares: [ 1  4  9 16 25 36]
Mean: 3.5
Transpose:
 [[1 4]
 [2 5]
 [3 6]]
```

Experiment / Notebook 16: 11.ipynb

Experiment 11: Pandas Library Use Cases

Executed notebook with output.

Code cell 2

```
import pandas as pd
df=pd.DataFrame({'Name':['Asha','Bala','Chitra','Dev'],'Department':['CSE','IT','CSE','IT'],'Marks':[82,75,91,68]})
print('DataFrame:'); display(df); print('Marks above 75:'); display(df[df.Marks>75]); print('Department averages:'); displa
```

Output 1

```
DataFrame:
```

Output 2

```
      Name Department  Marks
0    Asha        CSE     82
1    Bala         IT     75
2  Chitra        CSE     91
3     Dev         IT     68
```

Output 3

```
Marks above 75:
```

Output 4

```
      Name Department  Marks
0    Asha        CSE     82
2  Chitra        CSE     91
```

Output 5

```
Department averages:
```

Output 6

```
   Department  Marks
0         CSE   86.5
1          IT   71.5
```

Output 7

```
      Name Department  Marks Result
0    Asha        CSE     82   Pass
1    Bala         IT     75   Pass
2  Chitra        CSE     91   Pass
3     Dev         IT     68   Pass
```

Experiment / Notebook 17: 12.ipynb

Experiment 12: Outlier Detection

Executed notebook with output.

Code cell 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
df=pd.DataFrame({'Marks':[45,52,48,55,51,49,54,50,53,98]}); q1,q3=df.Marks.quantile([.25,.75]); iqr=q3-q1; lo,hi=q1-1.5*iqr,
```

Output 1

```
IQR bounds: 42.50 to 60.50
```

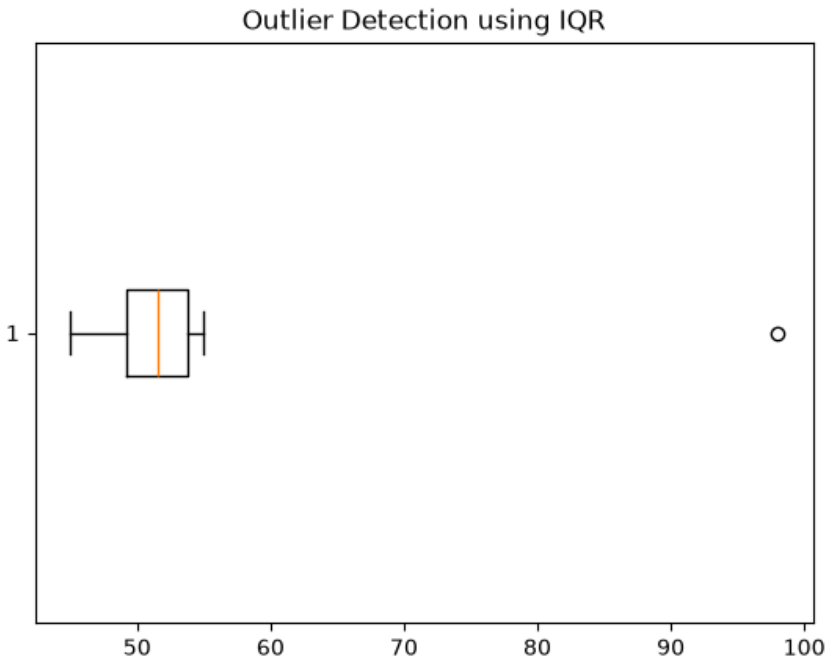
Output 2

```
   Marks  Outlier
0     45    False
1     52    False
2     48    False
3     55    False
4     51    False
5     49    False
6     54    False
7     50    False
8     53    False
9     98     True
```

Output 3

```
C:\Users\Pradeep\AppData\Local\Temp\ipykernel_35608\3651477238.py:5: MatplotlibDeprecationWarning: vert: bool was deprecated
  df=pd.DataFrame({'Marks':[45,52,48,55,51,49,54,50,53,98]}); q1,q3=df.Marks.quantile([.25,.75]); iqr=q3-q1; lo,hi=q1-1.5*io
```

Output image:



Output 4

```
<Figure size 640x480 with 1 Axes>
```

Experiment / Notebook 18: 13.ipynb

Experiment 13: Handling Missing and Inappropriate Data

Executed notebook with output.

Code cell 2

```
import numpy as np
import pandas as pd
df=pd.DataFrame({'Name':['Asha','Bala','Chitra','Dev'],'Age':[20,np.nan,-5,22],'Marks':[82,75,np.nan,91]}); print('Original
```

Output 1

```
Original:
```

Output 2

```
     Name   Age  Marks
0    Asha  20.0   82.0
1    Bala   NaN   75.0
2  Chitra  -5.0    NaN
3     Dev  22.0   91.0
```

Output 3

```
Cleaned:
```

Output 4

```
     Name   Age      Marks
0    Asha  20.0  82.000000
1    Bala  21.0  75.000000
2  Chitra  21.0  82.666667
3     Dev  22.0  91.000000
```

Output 5

```
Name     0
Age      0
Marks    0
dtype: int64
```

Experiment / Notebook 19: 14.ipynb

Experiment 14: Feature Scaling

Executed notebook with output.

Code cell 2

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler,StandardScaler
df=pd.DataFrame({'Age':[18,22,30,45,60],'Income':[18000,25000,42000,65000,90000]}); print('Original:'); display(df); print('
```

Output 1

```
Original:
```

Output 2

```
   Age  Income
0   18   18000
1   22   25000
2   30   42000
3   45   65000
4   60   90000
```

Output 3

```
Min-Max scaled:
```

Output 4

```
        Age    Income
0  0.000000  0.000000
1  0.095238  0.097222
2  0.285714  0.333333
3  0.642857  0.652778
4  1.000000  1.000000
```

Output 5

```
Standardized:
```

Output 6

```
        Age    Income
0 -1.093706 -1.130989
1 -0.836363 -0.867091
2 -0.321678 -0.226198
3  0.643356  0.640894
4  1.608391  1.583384
```

Experiment / Notebook 20: 15.ipynb

Experiment 15: Data Preprocessing

Executed notebook with output.

Code cell 2

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
df=pd.DataFrame({'City':['Chennai','Madurai','Chennai','Coimbatore'],'Age':[20,np.nan,25,30],'Income':[30000,45000,np.nan,60
```

Output 1

```
Raw data:
```

Output 2

```
          City   Age   Income
0     Chennai  20.0  30000.0
1     Madurai   NaN  45000.0
2     Chennai  25.0      NaN
3  Coimbatore  30.0  60000.0
```

Output 3

```
Preprocessed data:
```

Output 4

```
         Age    Income  City_Chennai  City_Coimbatore  City_Madurai
0 -1.414214 -1.414214             1                0             0
1  0.000000  0.000000             0                0             1
2  0.000000  0.000000             1                0             0
3  1.414214  1.414214             0                1             0
```

Experiment / Notebook 21: 16.ipynb

Experiment 16: EDA - Quantitative and Qualitative Analysis

Executed notebook with output.

Code cell 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
rng=np.random.default_rng(5); df=pd.DataFrame({'Department':rng.choice(['CSE','IT','ECE'],80),'Study_Hours':rng.normal(4,1.2
```

Output 1

```
Quantitative summary:
```

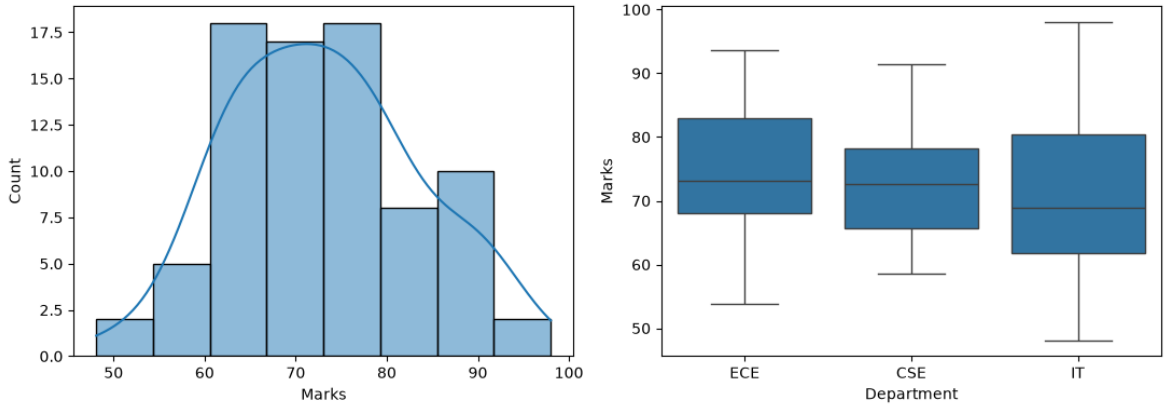
Output 2

```
       Study_Hours      Marks
count    80.000000  80.000000
mean      3.864480  73.105424
std       1.096305  10.440399
min       1.602620  48.104467
25%       3.098674  65.507307
50%       3.785723  72.344922
75%       4.601670  79.399617
max       6.918079  98.000000
```

Output 3

```
Qualitative frequency:
Department
CSE    33
ECE    25
IT     22
Name: count, dtype: int64
```

Output image:



Output 4

```
<Figure size 1100x400 with 2 Axes>
```

Experiment / Notebook 22: 17.ipynb

Experiment 17: Linear Regression

Executed notebook with output.

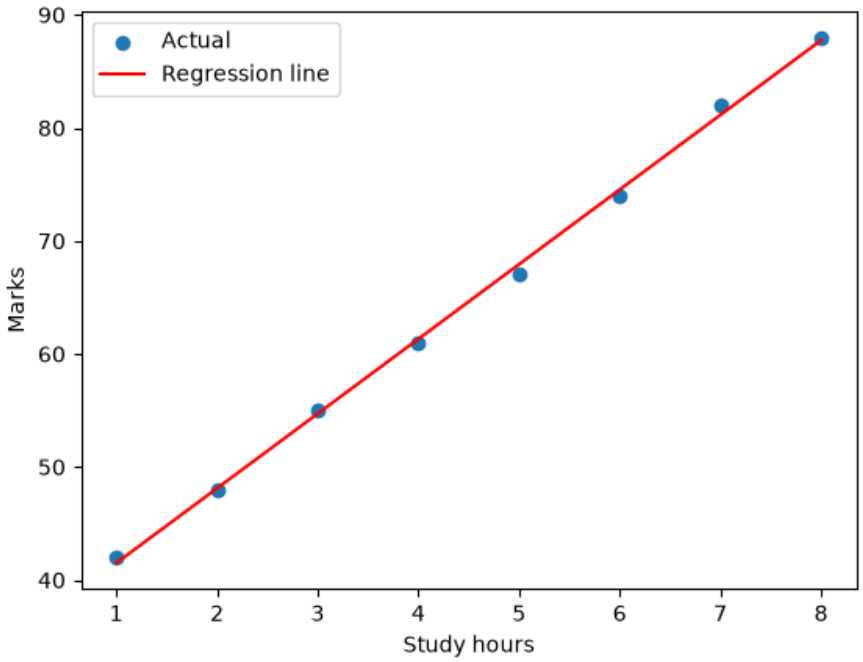
Code cell 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.linear_model import LinearRegression
hours=np.array([1,2,3,4,5,6,7,8]).reshape(-1,1); marks=np.array([42,48,55,61,67,74,82,88]); model=LinearRegression().fit(hou
```

Output 1

```
Equation: Marks = 6.61 x Hours + 34.89
Prediction for 6.5 hours: 77.84
```

Output image:



Output 2

```
<Figure size 640x480 with 1 Axes>
```

Experiment / Notebook 23: 18.ipynb

Experiment 18: KNN Algorithm

Executed notebook with output.

Code cell 2

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score,classification_report
d=load_iris(); Xtr,Xte,ytr,yte=train_test_split(d.data,d.target,test_size=.25,random_state=42,stratify=d.target); s=Standard
```

Output 1

```
Accuracy: 92.11%
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        12
  versicolor       0.81      1.00      0.90        13
   virginica       1.00      0.77      0.87        13

    accuracy                           0.92        38
   macro avg       0.94      0.92      0.92        38
weighted avg       0.94      0.92      0.92        38
```

Experiment / Notebook 24: 19.ipynb

Experiment 19: Logistic Regression

Executed notebook with output.

Code cell 2

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,confusion_matrix
d=load_breast_cancer(); Xtr,Xte,ytr,yte=train_test_split(d.data,d.target,test_size=.25,random_state=42,stratify=d.target); s
```

Output 1

```
Accuracy: 98.60%
Confusion matrix:
 [[52  1]
 [ 1 89]]
```

Experiment / Notebook 25: 20.ipynb

Experiment 20: K-Means Clustering

Executed notebook with output.

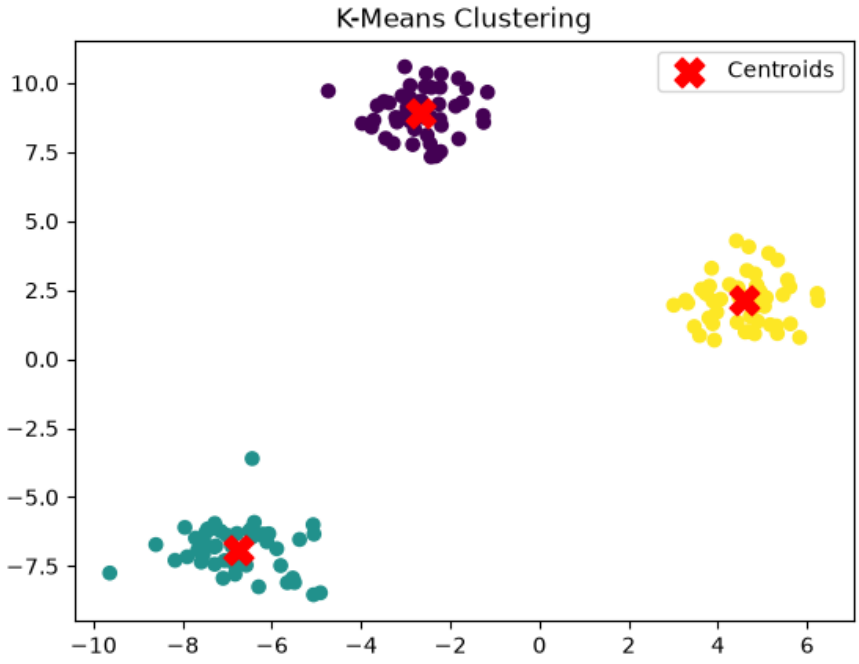
Code cell 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
X,_=make_blobs(n_samples=150,centers=3,cluster_std=.85,random_state=42); model=KMeans(n_clusters=3,n_init=10,random_state=42
```

Output 1

```
Cluster centers:
 [[-2.67  8.92]
 [-6.77 -6.89]
 [ 4.59  2.12]]
```

Output image:



Output 2

```
<Figure size 640x480 with 1 Axes>
```

Appendix: Repository CSV Data

sales_data.csv

Date	Product	Sales	Quantity	Region
01-01-2023	Product A	200	4	North
02-01-2023	Product B	150	3	South
03-01-2023	Product A	220	5	North
04-01-2023	Product C	300	6	East
05-01-2023	Product B	180	4	West
06-01-2023	Product A	210	5	North
07-01-2023	Product C	320	7	East
08-01-2023	Product B	160	3	South
09-01-2023	Product A	230	6	North
10-01-2023	Product C	310	7	East
11-01-2023	Product B	190	4	West
12-01-2023	Product A	240	6	North
13-01-2023	Product C	330	8	East
14-01-2023	Product B	170	3	South
15-01-2023	Product A	250	7	North
16-01-2023	Product C	340	8	East